MINERÍA DE DATOS · GUÍA DIDÁCTICA DE ESTUDIO

Algoritmo Apriori y Reglas de Asociación

> El algoritmo **Apriori** es una herramienta que sirve para encontrar **qué productos se compran juntos habitualmente**, permitiendo generar reglas medibles para orientar decisiones comerciales (como armar ofertas o acomodar góndolas).

1. Conceptos Base y Preparación de Datos

- **Diferencia con el Clustering (K-Means / DBSCAN):** Mientras que el clustering junta *clientes parecidos* (cada fila es una persona), Apriori junta *productos que coaparecen* (cada fila es un ticket o compra).
- **Transacción vs. Ítem vs. Itemset:**
  - **Transacción:** Es un ticket de compra único (identificado por `venta_id`).
  - **Ítem:** Un producto suelto (ej. `Leche`).
  - **Itemset:** Un grupo o conjunto de productos dentro de una compra (ej. `{Leche, Pan}`). No importa el orden ni las repeticiones (`{Leche, Pan}` es exactamente lo mismo que `{Pan, Leche}`).
- **La Matriz de Presencia (0 y 1):** A Apriori clásico **no le importa la cantidad** de unidades compradas. Si en un ticket llevaste 2 leches y 1 pan, el modelo solo se pregunta: *"¿Hay leche en el ticket? Sí (1). ¿Hay huevos? No (0)"*. Cada venta pesa una sola vez. Si te interesaran las cantidades o montos, tendrías que crear categorías aparte (discretizar), como por ejemplo un ítem llamado `Leche_2_o_más`.

2. Las 3 Métricas Clave (Las fórmulas del cuestionario)

Imaginá que analizamos un supermercado con un total de **10 tickets de venta**:

A. Soporte (Popularidad general)

Mide qué tan frecuente es un producto (o grupo de productos) sobre el **total absoluto de ventas**.

$$\text{Soporte}(X) = \frac{\text{Ventas que contienen X}}{\text{Total de ventas}}$$

- **Ejemplo individual:** Si el `Pan` aparece en 8 de los 10 tickets, su soporte es 8 / 10 = 80%.
- **Ejemplo de par:** Para `{Leche, Pan}`, no se suman sus soportes individuales; se cuentan los tickets donde **ambos aparecen juntos**. Si coinciden en 5 tickets de los 10, su soporte es 5 / 10 = 50%.

B. Confianza (Frecuencia condicional: Si lleva A, ¿lleva B?)

Una vez que tenés los conjuntos frecuentes, armás **reglas con dirección** (A → B). La confianza mide: *de todas las veces que alguien compró A, ¿qué proporción también llevó B?*

$$\text{Confianza}(A \rightarrow B) = \frac{\text{Soporte}(A \cup B)}{\text{Soporte}(A)}$$

- **Atención con la dirección:** Invertir la flecha cambia el resultado porque cambia el denominador.
  - `Leche -> Pan` : De las 6 compras con Leche, 5 tienen Pan (5 / 6 = 83,3%).
  - `Pan -> Leche` : De las 8 compras con Pan, 5 tienen Leche (5 / 8 = 62,5%).

C. Lift (¿Es una asociación real o pura casualidad?)

Una confianza alta puede ser engañosa. Si casi todo el mundo compra `Pan` (está en el 80% de los tickets), es obvio que quien compre `Leche` probablemente también lleve `Pan` por simple estadística, no porque haya una relación especial entre ambos. El **Lift** corrige esto comparando la confianza contra la popularidad general del segundo producto:

$$\text{Lift}(A \rightarrow B) = \frac{\text{Confianza}(A \rightarrow B)}{\text{Soporte}(B)}$$

- **Lift = 1:** No hay asociación (son independientes, se compran juntos por azar).
- **Lift > 1: Asociación positiva.** Aparecen juntos más de lo esperable (ej. `Pañales -> Toallitas` con Lift de 3,5 significa que llevar pañales multiplica por 3,5 la probabilidad normal de llevar toallitas).
- **Lift < 1: Asociación negativa.** Aparecen juntos menos de lo esperado (se "repelen" o son sustitutos, como pasa en el ejemplo con `Leche -> Queso` que da 0,833).
- **Propiedad clave:** A diferencia de la confianza, **el Lift es simétrico** (Lift(A → B) es igual a Lift(B → A)).

3. Cómo funciona el algoritmo: Niveles, Poda y Propiedad Apriori

Si tuvieras solo 20 productos, habría más de un millón de combinaciones posibles ($2^{20} - 1 = 1.048.575$). Para no calcular todo a lo bruto, Apriori filtra paso a paso usando un umbral llamado **Soporte Mínimo** (`min_support`):

1. **Nivel 1 (L_1):** Calcula el soporte de cada producto suelto. Si definís `min_support = 30%` (3 de 10 ventas), `Huevos` (20%) queda eliminado. Sobreviven `Pan` , `Leche` y `Queso` .
2. **La Poda (Propiedad Apriori):** Acá está el truco matemático del algoritmo. La regla dice: **"Si un conjunto no es frecuente, ningún conjunto más grande que lo contenga puede serlo"**. ¿Por qué? Porque agregar productos a un grupo solo puede achicar o mantener la cantidad de tickets que coinciden, nunca agrandarla. Como `{Huevos}` falló en el Nivel 1, el algoritmo **poda (descarta sin siquiera calcular)** cualquier combinación futura con huevos (como `{Huevos, Pan}` o `{Huevos, Leche, Queso}`).

3. **Nivel 2 (L_2) y Nivel 3 (L_3):** Arma parejas solo con los sobrevivientes (`{Leche, Pan}` , `{Leche, Queso}` , `{Pan, Queso}`). Como las tres superan el 30%, pasa a probar el trío `{Leche, Pan, Queso}` . El trío aparece en solo 2 tickets (20%), por lo que no alcanza el 30%.
4. **¿Cuándo frena el algoritmo?** Cuando ya no quedan candidatos frecuentes que superen el soporte mínimo para seguir armando grupos más grandes.

4. Trampas del Cuestionario (Errores frecuentes y Comparaciones)

Tené muy presentes estas aclaraciones conceptuales que suelen ser preguntas directas de examen:

Concepto / Frase	¿Verdadero o Falso?	Explicación Didáctica
"La regla A → B significa que el cliente agarró o escaneó primero A y después B"	Falso	No indica orden temporal ni de escaneo en la caja; es solo una proporción estadística condicional.
"Si hay una regla fuerte entre A y B, significa que comprar A causa la compra de B"	Falso	Asociación no es causalidad. Solo muestra que coaparecen en el ticket.
"Una confianza del 90% siempre garantiza una regla excelente"	Falso	Si el producto consecuente se vende en el 95% de los tickets, una confianza del 90% en realidad te da un Lift menor a 1. Siempre hay que mirar el Lift.
"Alcanza con mirar que el Lift sea muy alto"	Falso	Un Lift altísimo puede ser engañoso si está basado en muy poquitas ventas (soporte muy bajo). Necesitás equilibrio entre Soporte, Confianza y Lift.
"Apriori solo sirve para tickets de supermercado"	Falso	Si cada fila es un cliente y las columnas son sus características discretizadas (ej. `Zona_Norte` , `Rango_Edad_20_30`), sirve para ver qué atributos coaparecen.

- **Diferencia entre Apriori y FP-Growth:** Apriori es ideal para entender la lógica paso a paso (generando candidatos por nivel y podando), mientras que **FP-Growth** usa una estructura compacta que evita generar tantos candidatos intermedios, por lo que escala mejor cuando tenés bases de datos gigantescas.
- **Configuración de Umbrales:** Si ponés un `min_support` muy alto, perdés patrones interesantes poco comunes; si lo ponés muy bajo, se llena de candidatos basura y reglas inestables. En Python (`mlxtend`), esto se divide en dos funciones separadas: primero `apriori()` encuentra los conjuntos frecuentes según el soporte, y recién después `association_rules()` calcula las flechas, la confianza y el Lift.